

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R8
Mutable Data

Problems:

1. The following is the `make_withdraw` function shown in Lecture L8A:

```
def make_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw
```

Modify the `make_withdraw` function so that it creates password-protected accounts. That is, for example,

```
acc = make_withdraw(100, "my_password")
acc(30, "his_passcode") # returns "Wrong password; no withdraw"
acc(30, "my_password")  # returns 70
```

Moreover, to further increase security, we want an account to be disabled if it has been accessed three consecutive times with incorrect passwords. For example,

```
acc = make_withdraw(100, "my_password")
acc(30, "his_passcode") # returns "Wrong password; no withdraw"
acc(30, "my_password")  # returns 70
acc(10, "sesame")        # returns "Wrong password; no withdraw"
acc(15, "canola")        # returns "Wrong password; no withdraw"
acc(25, "olive")         # returns "Wrong password; no withdraw"
acc(30, "my_password")   # returns "Account disabled"
acc(30, "his_passcode")  # returns "Account disabled"
```

2. Write the function `reduce_list` that takes as arguments a binary function `op`, a value `init`, and a list `A`, and returns the value of $op(\dots(op(op(init, x_1), x_2))\dots, x_n)$, where $A = [x_1, x_2, \dots, x_n]$. Note that this works in the opposite order from the `reduce` function for **linked** lists (right-to-left folding). Example:

```
reduce_list(lambda x, y: x - y, 100, [1, 2, 3, 4, 5])
```

returns 85.

Note: Your function should not modify the input list `A`. In the rest of the course (including the next question), functions must not modify any of their arguments **unless explicitly specified**.

3. Write the function `filter_list` that takes as arguments a unary predicate function `pred` and a list `A`, and returns a new list that contains all elements of `A` that satisfy `pred`. Example: `filter_list(lambda x: x % 2 == 0, [1, 2, 3, 4, 5])` returns `[2, 4]`.